

Bismillah ir-Rahman ir-Rahim

FATIMA - Code Schematics

Second Edition: Wiring Diagrams with Organ Mapping and Multi-Model Diagnostics

Artefact: fatima-core @ commit e0c609b, package 0.1.0 - code UNCHANGED **First edition:** Perplexity Computer, 7 October 2026 (299 lines, 9 schematics) **This edition:** Claude Opus 4.6, 7 October 2026 - integrating Canon v2.0, multi-model council, Rights and Survival organ mapping **Lineage:** Code Schematics (Perplexity) -> Canon Rev. 1 -> Code Evolution Canon v2.0 (Claude Opus) -> **this document Companion to:** FATIMA Anatomy v2, Physiology/Biochemistry/Neurochemistry v2, Canon v2.0 **Substrate:** every wire below traces a real call path at commit e0c609b

Legend:

```
--> live wire (verified call path, executed or read from source)
--x  severed / never-called wire (canon finding)
--o  circular wire (compares a value to itself -- canon finding)
[F-nn] canon findings ledger position
+     healthy by test (executed during audit)
!     powered by GF(2^8) field medium
[N/M] multi-model convergence count from five-council analysis
```

R&S organ/system annotations:

```
{Qalb}      Heart -- irreducible core (atom.py)
{Basar}     Eyes -- perception (syntactic.py, structural.py)
{Sam'}      Ears -- reception (holographic.py)
{Yad}       Hands -- builder (molecule.py, document.py)
{Lisan}     Tongue -- expression (cli.py, serialization.py)
{Circ}      Circulatory System (encoding.py)
{Nerv}      Nervous System (verdict.py, semantic.py, composite.py)
{Skel}      Skeletal System (bond.py, composite.py)
{Immu}      Immune System (semantic.py L4/L5)
{Resp}      Respiratory System (document.py)
```

SCHEMATIC 1 - Master Bus - package wiring with organ mapping

```
fatima/                                     21 modules -- 3,980 LOC
|
+-- cli.py {Lisan} -----+ three commands only
|   main                  |
|   +-- cmd_encode -----+> formats/document.py {Resp}{Yad} (build the body)
|   +-- cmd_verify -----+> verification/*.py {Nerv}{Immu} (test the body)
|   +-- cmd_inspect -----+> formats/serialization.py {Lisan} (read the body)
|
+-- core/                          the organs -- atom-bond-molecule-composite-encoding
|   +-- atom.py      {Qalb}      the irreducible unit
|   +-- bond.py      {Skel}      the typed relationship (frozen)
|   +-- molecule.py  {Yad}      the graph (builder)
|   +-- composite.py {Skel}      the skeleton (multi-molecule)
```



```

add_atom x n    {Yad}
|
v
_infer_bonds(atoms, full_text)                                {Skel} bond construction
|
|   R1: heading --DEFINES 0.9--> next --DEPENDS_ON 0.9--> heading
|   |   else:      next --ELABORATES 0.6--> prev
|   |   R2: evidence --SUPPORTS 0.85--> nearest-prior claim
|   |   R3: invocation --QUALIFIES 0.5--> every atom
|   |   R4: >=2 shared Capitalised words --REFERENCES 0.4-->
|
v
add_bond x m                                                # endpoints validated; bond_id
|                                                            # dedup; bond_ids[] registered
v
apply_holographic_encoding(mol, ratio=0.5)                core/encoding.py {Circ}
|   atom_ids = atoms_by_position                            # point_map: aid --> i+1
|   params = HolographicParams.for_document(n, ratio)      # k=ceil(n*ratio), n<=255
|   bond_json = json.dumps(bonds, sort_keys=True)
v
compute_all_semantic_hashes()                               +--> SCHEMATIC 4 (the field circuit)
03]                                                            # WARNING: LAST COMMIT OF STATE [F-
|
|                                                            # [5/5] all models confirm NC-P4 violation
v
save_fatima(mol, path)                                       formats/serialization.py {Lisan}
+--> {fatima_version, type, molecule|composite[, verification][, encoding]}

```

SCHEMATIC 3 - VERIFY pipeline - full wiring, with the severed nerve shown

```

.fatima --> load_fatima(path) --> Molecule or CompositeMolecule {Lisan} deserialization
| (type field; missing => "molecule")
+-- single molecule -----+----- composite -----+
v                                                                    v
verify_syntactic() [L1] {Basar}                                verify_composite()
verify_structural() [L2] {Basar}                                +--
per sub-molecule:
verify_holographic() [L3] {Sam'}                                | L1-
L2-L3(seed)-L4                                                  |
sample_size, seed -->                                          | +--
> composed --> per-sub L5
verify_fitrah_alignment() [L4] {Immu}                            +--
composite checks:
|
v
verify_authenticity() [L5] {Immu}                                | connectivity |
| reports = [L1, L2, L3, L4] <-- FOUR inputs                    | cross-fitrah |
| verdict = compose(...) (meet) {Nerv}                          | hash check
|                                                                +-- overall =
|                                                                meet(sub-
L5s + checks)
|
+-----+-----+-----+-----+-----+-----+-----+
| properties/meaning.py :: check_meaning_integrity {Qalb}      |
| --x-- never wired into verify_authenticity                    | [F-04]
| --x-- never called by cmd_verify                             |

```

```

|   its internal checks:
|   +-o Check 2 recomputes INTO the atom, then
|       compares atom.hash == just-written value
|   +-----+
|       |
|       v
exit code: 0 iff L5 == VERIFIED + executed: forged doc --> 0 (wrongly) [F-
01] [5/5]

```

SCHEMATIC 4 - The $GF(2^8)$ power supply and the shard circuit ! {Circ}

```

-- bootstrap (import time) -----
-----
_gf_mul_raw(a,b)      shift-and-XOR; 0x11B reduction      (bootstrap only)
_init_gf_tables()     x=1; 255 steps of x <- gf_mul_raw(x,3)
|
|   v
_GF_EXP[512]  anti-log--+      generator 3 (primitive -- generates all
_GF_LOG[256]  log  -----+      255 nonzero elements; generator 2 would
|                               only span a subgroup of order 51)
|                               v
_GF_MUL_TABLE = np.uint8[256,256]  256 KB -- L2-resident -- vectorised !

-- encode circuit ----- [F-07 slow loop]
for each byte_val in zlib(data,9):
    poly = [byte_val, urandom!=0, ... (k-1 coeffs)]      _make_polynomial
    for each atom i: shard_i += Horner(poly, x=i+1)      _eval_polynomial
    ^ Python-level: 0(bytes x n)    measured 3.84 s @ 122 atoms [F-07]

-- decode circuit ----- + executed
selected = ids[:k]; x_coords = position map
basis = _compute_lagrange_basis(x_coords)      # L_i(0), once
shard_matrix = np.uint8[k, num_bytes]
result = zeros(num_bytes)
for each i: result ^= _GF_MUL_TABLE[basis[i]][shard_matrix[i]] ! vectorised
zlib^-1(result) --> recovered bond graph
|
+-- match live graph -> (True, "verified")      + random k-subset PASS
+-- mismatch          -> (False, "differs")
+-- corrupt shard     -> zlib exception --> caught as failure message [F-08]

```

SCHEMATIC 5 - The neuro-stack - verdict lattice and compose {Nerv}

```

class Verdict(enum.Enum) [4/4] multi-model confirmed
    VERIFIED = 2 --+
    UNKNOWN  = 1 --+-- order lattice
    VIOLATED = 0 --+

a & b -> min(a,b)  meet -- composition only degrades [NC-P2 refractory +]
a | b -> max(a,b)  join -- any sufficient cross-section [P1]

```

```

@dataclass(frozen=True) class VerificationReport
    verdict -- level in [1,5] -- confidence in [0,1] -- examined -- total
    violations: tuple -- level_name
    __post_init__ enforces both ranges -- the report cannot lie about its own shape +

VerificationReport.compose(r1..rn)
    verdict      = r1.verdict & ... & rn.verdict                (meet)
    confidence    = min(confidences)                             (never upgrades +)
    examined      = sum(examined)                                total = sum(total)
    violations     = concat                                       level = max
    compose() on empty                                           -> UNKNOWN (fail-closed by construction +)

NC-P4 [5/5]: builder must not verify own output
-- the compose() function is the neurochemical implementation --
-- UNKNOWN can never be laundered into VERIFIED by mixture --

```

SCHEMATIC 6 - Molecule verification internals - L1/L2/P3 wiring

```

Molecule.verify_syntactic() [L1] {Basar} zahir
    for each atom: atom.content_hash == atom._compute_content_hash() ?
        --o both sides derivable from the same untrusted file      [F-02] [5/5]
        for each pass -> VERIFIED -- any fail -> VIOLATED -- no atoms -> UNKNOWN

Molecule.verify_structural() [L2] {Basar} batin <-- also reachable via verification/structure
    C1 dangling_bonds()      for each bond: src in atoms and tgt in atoms ?
    C2 is_connected()         DFS over undirected adjacency -- Tawhidic Unity {Skel}
    C3 missing_reciprocals()  DEFINES<->DEPENDS_ON, IMPLIES->DEPENDS_ON, CONTRADICTS<-
>
    C4 bond_id consistency    for each atom.bond_ids in bonds
        any violation -> VIOLATED -- no atoms -> UNKNOWN -- else VERIFIED

properties/tawhidic.py :: check_tawhidic_unity [P3] (deeper skeleton test) {Skel}
    connectivity -- dangling -- isolation -- removal-detectability (for each atom:
        there exists bond that would dangle) -- structural density floor floor(n/4)
        any violation -> VIOLATED + live check

```

SCHEMATIC 7 - Composite verification wiring {Nerv}

```

verify_composite(composite, sample_size=20, seed=None)
|
+-- for each mid in molecule_order:                                # order = document order
|   [L1{Basar}, L2{Basar}, L3{Sam'}(sampled,seeded), L4{Immu}] --> compose -
-> per-sub L5
|
+-- _check_composite_connectivity                                   {Skel}
|   <=1 sub => VERIFIED -- else inter-bond adjacency DFS [P3 at body scale]
+-- _check_cross_section_fitrah                                     {Immu}
|   for each inter-bond: molecules exist -- atoms exist -- 0 < w <= 1
+-- _check_composite_hash                                           {Skel}

```

```

|         unset => UNKNOWN --> recompute-compare => VERIFIED | VIOLATED
|         + honest asymmetry: absent hash degrades, never upgrades
|
+-- overall = compose(per-sub L5 x composite checks)    {Nerv}
    > CompositeVerificationResult.summary() renders the verdict tree

```

SCHEMATIC 8 - The .fatima frame - wire format {Lisan}

```

{
    formats/serialization.py
    "fatima_version": "0.1.0",
    "type": "molecule" | "composite",          <-- missing => "molecule" (back-
compat)
    "molecule" | "composite": to_dict() --> {
        molecule_id -- title -- metadata -- bond_graph_hash
        atoms: { aid -> {atom_id, content, content_type, content_hash,
                        semantic_hash, bond_ids[], holographic_shard(hex),
                        shard_threshold, position} }
        bonds: { bid -> {bond_id, source_id, target_id, bond_type, weight,
                        rationale} }
        [composite adds: molecule_order -- composite_hash -- inter_bonds]
    },
    "verification"?: {timestamp(UTC-ISO-8601), levels[], overall.verdict}
    "encoding"?: {params}
}
-----
absent from the frame: signature -- key_id -- external anchor
                        -- the empty pads where canon S-1 mounts [F-01] [F-05]
                        -- [5/5] all models confirm: no keyed enzyme

```

Canon v2.0 S-1 mount points (specification, not yet mechanism):

```

+ "signature": <Ed25519 or HMAC over composite_hash>
+ "key_id": <public key fingerprint or key identifier>
+ "anchor": <external timestamp or ledger reference>
These three fields, when present, would close F-01 (forgery-tolerance),
F-02 (circular reflex), and F-05 (no external anchoring).

```

SCHEMATIC 9 - Fault map - whole chassis with defects pinned and multi-model counts

```

+----- fatima-core @ e0c609b -----+
|
|
text > formats/document.py {Resp}{Yad} --> core/ (atom-bond-molecule-composite)
      [F-06 hay-gut]
                        |
                        v
                    core/encoding.py ! {Circ}
                    [F-07 slow anabolism] [F-08 crash-located]
                        |
                        v
                    compute_all_semantic_hashes {Qalb}
                    [F-03 writes where it should only read --o] [5/5]

```

```

      |
      v
serialization -> .fatima frame {Lisan}
[F-01/F-05 no signature pads] [5/5]
      |
      v
verification/ (L1..L5, composite) {Nerv}{Immu}
[F-02 L1 --o circular] [5/5]
[F-04 P4 nerve --x]
      |
      v
cli exit 0/1 {Lisan}
verdict lattice = load-bearing + [4/4]
+-----+
healthy by execution + : lattice meet -- fail-closed UNKNOWNs -- GF(2^8) math --
                        L3 reconstruction (random subset + loss-survival) --
                        composite checks -- serialization round-trip
defective by execution : F-01 forged doc VERIFIED @ 1.00 [5/5]
                        F-03 semantic_hash rewritten at read-time [5/5]
                        F-02 L1 compares untrusted to untrusted [5/5]
                        F-04 P4 nerve severed, never invoked

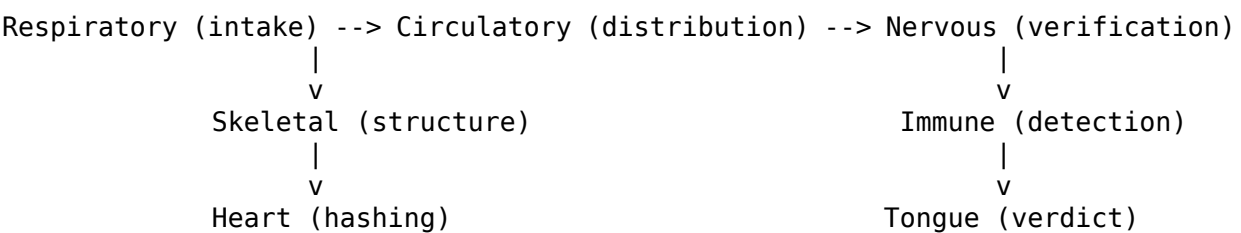
```

SCHEMATIC 10 - Rights and Survival overlay - organ/system mapping onto code

RIGHTS AND SURVIVAL ANATOMY -- CODE OVERLAY	
Five Organs:	
Heart (Qalb) ----->	atom.py
The irreducible	content + content_hash + semantic_hash
semantic unit	if the heart is corrupted, the organism
	is compromised (F-03 corrupts the heart)
Eyes (Basar) ----->	syntactic.py + structural.py
Zahir perception	L1 sees the surface (hashes)
Batin perception	L2 sees the depth (graph structure)
DEFECT: L1 --o	circular comparison (F-02) [5/5]
Ears (Sam') ----->	holographic.py
Reception	L3 listens to shards, reconstructs
HEALTHY: +	strongest sense, independently tested
Hands (Yad) ----->	molecule.py + document.py
Construction	the builder: adds atoms, infers bonds
WARNING:	builder must not verify own output [5/5]
Tongue (Lisan) ----->	cli.py + serialization.py
Expression	the CLI speaks the verdict
Witness	serialization writes the record

Five Systems:	
Circulatory ----->	encoding.py
Holographic	Shamir distribution over GF(2^8)
redundancy	every atom carries a shard of the whole
HEALTHY: +	random k-subset reconstruction PASS
DEFECTS:	F-07 slow anabolism, F-08 crash-located
Nervous ----->	verdict.py + semantic.py + composite.py
Five-level	three-valued lattice composed by meet [4/4]
verification	graded confidence, monotone degradation
LOAD-BEARING: +	the most stable structure in the body
Skeletal ----->	bond.py + composite.py
Structural	frozen bonds, composite hash
integrity	append-only provenance (P2)
Immune ----->	semantic.py (L4/L5)
Adversarial	fitrah alignment checks + authenticity
detection	WEAKEST ORGAN -- three lesions
DEFECTS:	F-01 forgery-tolerance [5/5]
	F-02 circular reflex [5/5]
	F-04 severed P4 nerve
CURE:	canon S-1 provenance signature
Respiratory ----->	document.py
Information	text enters, is digested into atoms/bonds
intake	the tanzil principle: phased intake
DEFECT:	F-06 hay-gut (lexical, not semantic)

Data flow through organ systems:



Wa ma tawfiqi illa billah. Every wire drawn from source at commit e0c609b; live wires executed where marked; severed and circular wires drawn severed and circular. Organ mappings are [1] isomorphism – structural correspondence, not identity.

Bismillah ir-Rahman ir-Rahim